Contents

Cardiovascular Disease Prevention Platform

Project Information

Field	Value
Student	Darya Korbut
Group	JS/TS
Supervisor	Alevtina Gourinovich
Date	05.01.2026

Links

Resource	URL
Production	[Deployed Application URL]
Repository	https://github.com/Doshirac/cvd-platform
API Docs	https://app.gitbook.com/invite/YJjvuHTqbLlvmjZEzQ
Design	https://docs.google.com/document/d/19rmvxAH_jqToa DLcIrTWUuQ/edit?usp=sharing

Elevator Pitch

A Cardiovascular Disease Prevention Platform is a web-based library that curates cardiovascular diseases into standardized "cards" with consistent descriptions, primary vs. secondary (symptom-related) factors, and practical prevention guidance. It is designed for clinicians and medical educators who need reliable, structured reference material, and for public-health analysts who need comparable prevalence and trend indicators from trusted datasets. The platform solves the problem of fragmented, inconsistently structured CVD information by unifying content and analytics in one place, reducing time spent reconciling sources and improving clarity in prevention-focused decisions. What makes it unique is the combination of standardized disease entries with per-disease

analytics and explicit separation of primary symptoms from secondary factors, plus an API-first foundation that can scale beyond the diploma MVP (including English/Russian UI support).

Evaluation Criteria Checklist

#	Criterion	Status	Documentation
1	Frontend Architecture & UX Delivery		02-technical/criteria/frontend.md
2	Backend Architecture & API Delivery		02-technical/criteria/backend.md
3	Database Design & Data Integrity		02-technical/criteria/database.md
4	Observability (Logging + Error Monitoring)		02-technical/criteria/observability.md
5	Research Pipeline & Analytics Artifacts		02-technical/criteria/research.md
6	Deployment & Local Environment		02-technical/deployment.md
7	API Contract & Reference Documentation		appendices/api-reference.md

Documentation Navigation

- Project Overview - Business context, goals, and requirements
- Technical Implementation - Architecture, tech stack, and criteria details
- User Guide - How to use the application
- Retrospective - Lessons learned and future improvements

Document created: 05.01.2026 Last updated: 05.01.2026

1. Project Overview

This section covers the business context, goals, and requirements for the project.

Contents

- Original Documentation
- Problem Statement & Goals

- Stakeholders & Users
- Scope
- Features

Executive Summary

This project delivers a web platform that consolidates fragmented cardiovascular disease (CVD) information into a standardized, referenced disease library. It targets clinicians/educators and public-health analysts who need fast lookup with transparent sources. The solution is a React SPA backed by an Express + Prisma + PostgreSQL API, with bilingual UI (EN/RU), search, filters, and a references/research area.

Key Highlights

- Problem: inconsistent structure and weak source transparency in CVD info.
- Solution: standardized disease cards with references + lightweight dataset-driven analytics.
- MVP outcomes: 25 disease cards; 2 datasets with provenance (source + last updated).

2. Technical Implementation

This section describes how the CVD platform is implemented: the frontend SPA, the backend REST API, the database schema, the research/analytics workflow, and the deployment/runtime setup.

Contents

- Tech Stack
- Criteria Documentation - ADRs for evaluation criteria
- Deployment

Solution Architecture

High-Level Architecture

Diagram - High-level architecture diagram

Additional diagrams: - ER Diagram - Deployment Diagram

System Components

Component	Description	Technology
Frontend	Single-page application (disease library UI, theming, routing, error boundaries).	React + TypeScript + Vite
Backend	REST API for diseases/symptoms/risk factors/sources, pagination/filtering, locale-aware responses, OpenAPI docs.	Node.js + TypeScript + Express
Database	Persistent, normalized content store (diseases, symptoms, risk factors, translations, junction tables).	PostgreSQL 16 + Prisma
Cache	Optional read caching to reduce DB load for frequently accessed data.	Redis 7 (ioredis client)
External Services	Error monitoring and performance tracking (optional via env).	Sentry

Data Flow

```
[User Action] → [Frontend (React)] → [HTTP Request] → [Backend (Express)]

                                                    [Cache (Redis)]  (hit)   [Response]

                                                    (miss) [Prisma] → [PostgreSQL] → [R

[UI Update] ← [Frontend renders data] ←
```

Locale-aware content is handled by selecting translations (EN/RU) at the API/service layer based on the `locale` query parameter.

Key Technical Decisions

Decision	Rationale	Alternatives Considered
React + Vite SPA	Fast iteration and simple deployment for a content-focused UI.	Next.js SSR/ISR

Decision	Rationale	Alternatives Considered
Express + Prisma API	Lightweight REST API with type-safe DB access and migrations.	Fastify, NestJS, Python (FastAPI/Django)
PostgreSQL normalized schema + translation tables	Data integrity, M:N modeling (with metadata), and bilingual support.	Single-table MVP, MongoDB
Redis caching for read-heavy endpoints	Improves latency and reduces DB load for repeated requests.	No cache, in-memory cache
Observability via Sentry + structured logs	Practical debugging and error reporting without heavy APM tooling.	Console-only logs, full APM suites

Criteria Documentation

The evaluation criteria ADRs are located in criteria/ and include:

- Frontend
- Backend
- Database
- Research
- Deployment
- Observability
- Testing

3. User Guide

This chapter is a practical guide to using the CVD Platform as an end user (read-only browsing) and as a reviewer running it locally.

Contents

- Features Walkthrough
- FAQ & Troubleshooting

Getting Started

System requirements

Requirement	Minimum	Recommended
Browser	Chrome / Edge / Firefox / Safari (latest 2 major versions)	Latest version
Screen Resolution	1280×720	1920×1080 or higher
Internet	Required for external source links and API-backed content	Stable connection
Device	Desktop or mobile (responsive UI)	Desktop for most comfortable reading

Accessing the Application

1. Open your web browser
2. Navigate to the application URL:
   - **Production:** (if deployed) use your deployed URL
   - **Local development:** start the frontend and open `http://localhost:5173`
3. If you are running locally, ensure the API is available at `http://localhost:4000/api`

Run locally (recommended)

1. Start the stack with Docker Compose (API + DB + Redis).
2. Seed the database.
3. Start the frontend dev server.

Expected local URLs:

- Frontend: `http://localhost:5173`
- API: `http://localhost:4000/api`
- Swagger: `http://localhost:4000/api-docs`

First launch checklist

1. Open the app URL and verify the header/navigation renders.
2. Toggle theme (light/dark) and confirm it persists on refresh.
3. Browse the disease list, open at least one disease entry, and verify loading/empty states are understandable.

Quick start (most common tasks)

- Browse diseases: open **Home** and scroll the list.
- Search/filter: use the page controls (if present). If testing the API directly, use `GET /api/diseases` with `search`, `symptom`, `riskFactor`, `skip`, `take`, `locale`.
- Verify sources: open **Sources** and follow the external resource link.
- Check system health: `GET /api/health` (liveness) and `GET /api/health/details` (details).

If the UI shows "Not Found" for a linked page, treat it as an implementation gap in the current build and continue testing via available pages or the Swagger UI.

Verifying via API (optional)

If you need to validate functionality without relying on the UI wiring, use Swagger (`/api-docs`) or call the endpoints directly:

- Health: `GET /api/health/details`
- Diseases list: `GET /api/diseases?take=10&skip=0&locale=en`
- Search: `GET /api/diseases?search=heart&locale=en`
- Sources: `GET /api/sources?take=10&skip=0`

Roles

- Visitor/Reader: read-only browsing, theme/language preference.
- Developer/Reviewer: runs services locally, seeds DB, verifies endpoints via Swagger (`http://localhost:4000/api-docs`).

4. Retrospective

This section reflects on the project development process, lessons learned, and future improvements.

What Went Well

Technical Successes

- The backend architecture (controllers → services → repositories) with Prisma + PostgreSQL provided a clear structure for read-heavy endpoints and helped keep API behavior predictable.
- Docker + Docker Compose made local development reproducible (API + DB + Redis), which simplified troubleshooting and allowed the project to be run consistently across machines.
- The normalized data model with translation tables (EN/RU) and junction tables for symptoms/risk factors supported the project's main academic

goal: standardized "disease cards" with explicit primary vs secondary factors.
- Observability was set up early (Winston/Morgan on the backend and Sentry hooks), which helped reduce time spent debugging silent failures.
- Testing infrastructure exists on both sides (backend unit tests with Jest; frontend unit tests + Playwright e2e), which provides a foundation for improving reliability.

Process Successes

- The scope was kept focused on a read-only MVP (no user accounts, no PHI/PII processing), which reduced compliance risk and kept the system design defensible for a thesis project.
- Documentation was treated as a deliverable: API reference, DB schema, criteria/ADR-style pages, and the user guide were aligned to the actual codebase instead of staying as templates.
- Incremental development worked well: backend/data model + docs were stabilized first, then the user-facing flows were documented and validated against repository reality.

Personal Achievements

- Built and maintained an end-to-end stack (React + Vite frontend, Express + Prisma backend, PostgreSQL + Redis, Docker Compose) within a thesis timeline.
- Improved skills in data modeling and schema evolution (Prisma migrations, seed strategy, normalized entities + translations).
- Strengthened the ability to translate academic requirements into an implementable MVP (scope control, out-of-scope decisions, and criterion-based documentation).

What Didn't Go As Planned

Planned	Actual Outcome	Cause	Impact
Fully working frontend pages for the core routes (Home, Sources, Disease details, Research cards)	Frontend is partially scaffolded; some routes/pages are not fully wired and users may reach Not Found for intended navigation paths	Small amount of time and the teachers sent the requirements to the frontend after the deadline, so the frontend could not be completed	Critical
Standardized "Disease Card" page (description, symptoms, prevention + references)	Backend/data model support exists, but the full disease card UI flow is not finished end-to-end in the current build	Time constraints; prioritization of backend + documentation completeness for thesis delivery	High
Fully functional localization toggle affecting content (EN/RU)	Locale support exists in the backend and data model; the UI language switch behavior may be incomplete depending on current wiring	Limited time for implementing full i18n flow across pages and ensuring consistent API requests with `locale`	Medium

Challenges Encountered

1. **Late / shifting requirements for frontend delivery**
   - Problem: Frontend requirements were clarified late, after the planned deadline.
   - Impact: The UI could not be completed to the same level as the backend/data model and documentation, and some routes remain scaffolded rather than finished.

- Resolution: Focused on stabilizing the backend, database schema, and documentation so the project remains coherent and defensible; documented the current UI limitations transparently.

2. **Keeping documentation aligned with real implementation**
   - Problem: Several documentation files started as templates and could easily drift away from the actual code behavior.
   - Impact: Risk of thesis documentation being inconsistent with the system (especially around endpoints, error responses, and "no results" responses).
   - Resolution: Re-checked controllers/OpenAPI and updated the documentation to match the implemented API responses and the current frontend wiring status.

Technical Debt & Known Issues

ID	Issue	Severity	Description	Potential Fix
TD-001	Frontend routes/pages not fully wired	High	Navigation can include intended routes, but some route targets are not fully implemented yet, leading to Not Found for users.	Finish route configuration and implement missing pages; add basic smoke/e2e coverage for the primary flows.
TD-002	Inconsistent instantiation / DI usage in backend	Medium	Some DI patterns are present but not consistently applied across all setup paths, which can make maintenance harder as the project grows.	Standardize one approach (DI container everywhere or explicit wiring) and remove partial patterns.
TD-003	API "no results" semantics may surprise clients	Low	Some list endpoints return HTTP 200 with `{ "message": "No disease found." }` instead of an empty array; this requires special handling in the UI/client.	Standardize list responses (prefer empty arrays + metadata) or document + wrap the behavior consistently in the frontend API layer.

Code Quality Issues

- Frontend feature pages and router wiring need consolidation (ensure each header link maps to an implemented route and consistent layout state).
- Improve end-to-end coverage for primary user flows (browse diseases → open disease card → view sources), so regressions are caught early.
- Reduce duplication between built JS output (build folder) and TypeScript sources when referencing code in documentation and ensure a single source of truth during development.

Future Improvements (Backlog)

If there was more time, these features/improvements would be prioritized:

High Priority

1. **Finish core frontend flows (MVP completeness)**
   - Description: Implement and wire the main routes (Home, Sources, Disease details, Research cards) to the real backend endpoints; ensure consistent loading/empty/error states.
   - Value: Delivers the end-user experience described in the thesis and removes current Not Found gaps.
   - Effort: Medium–High
2. **Implement robust localization behavior end-to-end**
   - Description: Ensure the language switch reliably controls `locale` in API requests and updates visible content across pages.
   - Value: Meets the bilingual requirement and improves usability for the target audience.
   - Effort: Medium

Medium Priority

3. **Standardize API response contracts and client handling**
   - Description: Make list endpoints return consistent shapes (arrays + optional pagination metadata) and normalize behavior in the frontend API client.
   - Value: Simplifies UI integration and reduces edge-case bugs.
4. **Strengthen testing for user-facing behavior**
   - Description: Add Playwright coverage for "happy path" navigation and empty-state flows; expand backend integration tests for filtering/search.
   - Value: Prevents regressions and improves confidence during final thesis demos.

Nice to Have

5. Add basic SEO-friendly SPA metadata (title/description per route) and a sitemap for improved discoverability.
6. Improve caching strategy (TTL tuning and invalidation plan) for read-heavy endpoints.
7. Formalize the analysis pipeline (pinned Python env, repeatable scripts, artifact export conventions).

Lessons Learned

Technical Lessons

Lesson	Context	Application
A normalized schema pays off even for MVPs	Symptoms/risk factors and bilingual content quickly become complex without structure	Start with a clean relational model and add only required entities/relations, but avoid over-denormalization early.
Standardized error/empty-state contracts matter	Clients must handle "no results" and error responses consistently	Treat response shapes as part of the product contract; validate with docs and tests.
Reproducible environments reduce project risk	Docker Compose allowed fast setup and fewer "works on my machine" issues	Containerize early and document the environment variables and health checks.

Process Lessons

Lesson	Context	Application
Requirements need a freeze point	Late frontend requirements significantly reduced UI completeness	Agree on an MVP requirement freeze date and re-scope explicitly when changes arrive late.
Documentation is easier when written alongside implementation	Template docs drift quickly if not updated continuously	Keep docs close to code changes and validate against controllers/OpenAPI and UI behavior.

What Would Be Done Differently

Area	Current Approach	What Would Change	Why
Planning	Backend + docs stabilized first; UI completed later	Set explicit milestones for "end-to-end MVP demo" earlier (even if the UI is minimal)	Prevents a situation where backend is strong but UI lags due to late work.
Technology	SPA + REST + Prisma + Docker	Keep the stack, but enforce one wiring approach (DI vs explicit composition)	Reduces maintenance overhead and avoids partial architectural patterns.
Process	Some requirements clarified late	Add a formal requirements sign-off checkpoint with stakeholders	Reduces last-minute changes and scope creep.

Area	Current Approach	What Would Change	Why
Scope	Ambitious feature list early	Cut more aggressively to "core reading flows" first, then extras	Improves the probability of a complete user experience within a fixed academic deadline.

Personal Growth

Skills Developed

Skill	Before Project	After Project
Full-stack system delivery (FE/BE/DB)	Intermediate	Advanced (clearer layering, better integration discipline)
Database modeling + migrations (Prisma/PostgreSQL)	Beginner–Intermediate	Advanced (normalized schema, migrations, seeding strategy)
Testing + quality practices (Jest/Playwright/Storybook)	Beginner	Intermediate (foundation in place; clear next steps for coverage)

Key Takeaways

1. A coherent MVP requires an end-to-end "happy path" early, not only strong backend pieces.
2. Data modeling decisions (translations, junction tables, enums) strongly determine how maintainable the product becomes.
3. Clear contracts (API responses, empty/error states, and documented limitations) reduce confusion and make thesis evaluation easier.

Retrospective completed: 2026-01-05

Features & Requirements

Epics Overview

Epic	Description	Stories	Status
E1: Browse Disease Library	Discover diseases via a simple list view.	2	
E2: Search Diseases	Find diseases by name/keywords (full-text).	1	
E3: Filter by Category	Narrow results by category/risk/severity.	1	
E4: View Disease Card	Read a standardized disease card (description, symptoms, prevention).	1	
E5: View Research Cards	View research content on the website via read-only research cards.	1	
E6: Switch Language EN/RU	Toggle English/Russian on core pages.	1	
E7: View References	View sources/guidelines/datasets and per-card references.	1	

User Stories

Epic 1 — Browse Disease Library

First entry point for discovery: a disease list that loads quickly and links to disease cards.

ID	User Story	Acceptance Criteria	Priority	Status
US-001	As a User, I want to see a list of cardiovascular diseases, so that I can quickly find conditions to read.	- List loads 2s with 25 diseases- Each item links to its card- Clear empty/error states	Must	
US-002	As a User, I want to sort diseases (A–Z / Z–A), so that I can navigate faster.	- Sort applies instantly- Sort persists in session	Could	

Epic 2 — Search Diseases

Header search input for direct lookup with a no-results state.

ID	User Story	Acceptance Criteria	Priority	Status
US-003	As a User, I want to search by disease name or keywords, so that I can find a specific disease.	- Results appear $<$ 1s on typical data- Highlight matches- No-results state shown	Must	

Epic 3 — Filter by Category

Faceted discovery: filters that update results and can be shared via URL.

ID	User Story	Acceptance Criteria	Priority	Status
US-004	As a User, I want to apply filters, so that I can narrow the list to what I need.	- Filters update results instantly- “Clear all” resets state- URL reflects active filters (shareable)	Must	

Epic 4 — View Disease Card

Core reading page with standardized sections and references.

ID	User Story	Acceptance Criteria	Priority	Status
US-005	As a User, I want to open a disease card, so that I can read description, primary/secondary symptoms, and prevention.	- Sections visible (Description; Primary vs Secondary symptoms; Prevention)- Loads 2s (analytics may lazy-load)- Sources/references visible	Must	

Epic 5 — View Research Cards

Read-only research cards that summarize research content on the website.

ID	User Story	Acceptance Criteria	Priority	Status
US-006	As a User, I want to view research cards, so that I can quickly read research summaries on the website.	- Research cards list is available on the Research page- Each card shows title + short summary- Clear empty/error states	Should	

Epic 6 — Switch Language EN/RU

Language selection for UI labels and core content.

ID	User Story	Acceptance Criteria	Priority	Status
US-007	As a User, I want to switch between English and Russian, so that I can read in my preferred language.	- UI labels and content switch- Choice persists across pages/sessions	Must	

Epic 7 — View References

Dedicated references view with transparent data provenance.

ID	User Story	Acceptance Criteria	Priority	Status
US-008	As a User, I want to see sources and datasets, so that I can verify the information.	- List shows title, publisher, link, license, last-updated- Links open in a new tab- Per-card source list available	Should	

MoSCoW Summary

- **Must Have:** EP1 (US-001), EP2 (US-003), EP3 (US-004), EP4 (US-005), EP6 (US-007)
- **Should Have:** EP5 (US-006), EP7 (US-008)
- **Could Have:** EP1 (US-002)
- **Won't Have (MVP):** user accounts, comments, ML risk scores, real-time dashboards, native apps

Use Case Diagram

Diagram - Here you can find the diagram

Non-Functional Requirements

Performance

Requirement	Target	Measurement Method
Page load time	Main pages LCP < 2.5 s on typical 4G devices	Lighthouse (mobile throttling), WebPageTest (optional)
API response time	Core API p95 < 300 ms on core endpoints (local/prod-like)	k6 / autocannon against `/diseases` and `/sources`
Concurrent users	Stable behavior under typical MVP load (e.g., 25–50 concurrent)	Load test script + error-rate monitoring

Security

- **HTTPS** in production environments.
- **Basic rate limiting** on public endpoints to mitigate abuse.
- **Security & Privacy:** no PII/PHI stored; platform is read-only.
- **Input validation** on API query parameters; safe error responses.
- **Secrets/config** provided via environment variables (env-based configuration).

Accessibility

- Target: no critical accessibility blockers on core flows.
- Keyboard navigation for primary interactions; visible focus states.
- Clear labels/ARIA for interactive controls (search, filters, toggles).
- Glossary and medical disclaimer available for clarity.

Scalability

- Stateless API suitable for horizontal scaling.
- Cached assets (e.g., pre-rendered SVG charts) for efficient delivery.

Usability & Localization

- UI available in **English (default)** and **Russian** on all core pages.
- Responsive UI: works correctly on desktop, tablet, and phone.
- Basic **Dark/Light theme toggle**.

Data Quality

- Each indicator shows **source** and **last updated**.
- Data lineage is maintained via the References page and dataset attribution.

Observability

- Structured logs and simple metrics (latency, error rate).
- Container health checks to support basic monitoring.

Docs & Deployment

- OpenAPI/Swagger documentation available for core endpoints.
- Docker Compose setup for local/prod-like environment.
- Environment-based configuration.

Reliability

Metric	Target
Uptime	Best-effort for MVP (non-critical system)
Recovery time	Restore service from backup in 60 minutes (target)
Data backup	Automated daily backups; verified by a successful restore test

Compatibility

Platform/Browser	Minimum Version
Chrome	Latest 2 major versions
Firefox	Latest 2 major versions
Safari	Latest 2 major versions
Mobile	iOS 16+ / Android 11+

Regulatory/Compliance Needs

- **GDPR (EU):** No PII/PHI processed (read-only library). Publish a Privacy Notice stating data minimization; if analytics/cookies are added, use explicit consent and provide opt-out.
- **ePrivacy / Cookie Rules:** Show a cookie banner only if using non-essential cookies (e.g., analytics). Store consent choices.

- **Medical Disclaimer:** Clear statement that content is informational, not medical advice, not for diagnosis or treatment; advise consulting a physician.
- **Medical Device Regulation (EU MDR):** Not applicable (no diagnostics/triage/personal recommendations). Include an MDR non-applicability note.
- **Data Licensing & Attribution:** Respect open-data licenses (WHO/EU/national). Display source + license + last updated on indicators and in the References page.
- **Copyright / Content Use:** Ensure rights to text, images, SVG charts; attribute third-party materials per license; avoid logos/marks without permission.
- **Terms of Use:** Publish concise Terms covering permitted use, no warranty, and liability limitations.
- **Open-Source Compliance:** Track dependencies and their licenses; include notices as required.

Problem Statement & Goals

Context

This thesis project is a read-only CVD knowledge platform for education and public-health exploration. It focuses on evidence-linked summaries and lightweight dataset-driven indicators (not diagnosis or clinical decision support).

Vision Statement

Build a clear, reliable platform with standardized cards for cardiovascular diseases (description, symptoms, prevention) and basic statistics such as prevalence and key indicators from trusted data sources. The platform helps clinicians, educators, and public-health analysts distinguish primary symptoms from secondary factors, and it is designed to be extendable beyond the diploma scope.

Problem Statement

Who: Clinicians, medical educators, public-health analysts, and readers who need structured, referenced CVD information and basic analytics.

What: Cardiovascular disease information is fragmented, inconsistently structured, and often lacks analytical context (prevalence/indicator breakdowns) and clear separation of primary symptoms vs. secondary (symptom-related) factors.

Why: Clinicians and educators spend time reconciling disparate sources; analysts lack a unified, transparent view of prevalence and indicators; and readers struggle to interpret symptoms vs. risk/prevention factors. This leads to slower information retrieval, unclear prevention guidance, and missed insights.

Pain Points

#	Pain Point	Severity	Current Workaround
1	Information is scattered across many sources and not standardized (terminology, structure, completeness).	High	Manual search in guidelines/articles; personal notes; ad-hoc comparisons.
2	Primary symptoms vs. secondary factors (risk/prevention-related) are often mixed, making interpretation and teaching harder.	High	Rewriting content into personal templates; discussing with colleagues; cross-checking multiple sources.
3	Lack of a single place to read curated research summaries and supporting context linked to transparent sources.	Medium	Searching publications manually; bookmarking links; assembling summaries in notes.
4	Unclear data provenance (source, last updated, licensing) reduces trust and reusability.	Medium	Checking dataset websites and PDFs manually; storing links outside the workflow.
5	Hard to quickly find relevant diseases and compare them without strong search/filters.	Medium	Browser search; bookmark lists; manual scanning.

Business Goals

- Provide 25 standardized disease cards with references.
- Make discovery fast (search + filters) and bilingual (EN/RU).
- Publish a references area with dataset provenance (source + last updated).

- Keep the system reproducible and maintainable (Docker, migrations/seed, OpenAPI).
- No PII/PHI processing.

Objectives (MVP Targets)

- Content: 25 disease cards.
- Data: 2 official datasets integrated with provenance.
- UX: search + filters + EN/RU toggle on core pages.
- API: documented core endpoints via OpenAPI.

Non-Goals

- No diagnosis/treatment advice or symptom checker.
- No user accounts.
- No real-time BI stack or ML prediction.
- No native apps/offline mode.

Project Scope

In Scope

- Disease library (25 entries) with standardized content and references.
- Search and filters for discovery.
- EN/RU locale support.
- Sources/references page with provenance (source + last updated).
- Optional research summaries/cards (read-only).
- Platform: React frontend + Express/Prisma backend + PostgreSQL + Docker + OpenAPI.

Out of Scope

Feature	Reason	When Possible
Medical diagnosis/treatment advice, triage, or symptom checker	Medical decision-making is not the goal of the academic platform and would add clinical/compliance risk.	Future phase / Never

Feature	Reason	When Possible
User accounts for readers (bookmarks, comments, forums)	Requires identity, moderation, and data protection that exceeds the MVP scope.	Future phase
Advanced BI stack (Power BI/Fabric), real-time streaming dashboards	Adds complexity and infrastructure cost; not required for thesis MVP goals.	Future phase
ML risk prediction or personalised recommendations	Requires model development, validation, and additional governance.	Future phase
Native mobile apps (iOS/Android) and offline mode	Requires separate platform build and maintenance; not needed for MVP.	Future phase
Paid/proprietary datasets or manual data entry from paywalled sources	Conflicts with budget constraint and licensing limitations.	Never

Assumptions

#	Assumption	Impact if Wrong	Probability
1	Data availability & licensing: official datasets are publicly accessible and permit academic use with citation.	Data ingestion scope would be reduced; alternative datasets/sources required.	Medium
2	Stability of APIs/files: data formats won't change materially during the build.	Additional work to adapt ETL/parsing; schedule risk.	Medium

#	Assumption	Impact if Wrong	Probability
3	No personal/medical data (PII/PHI) is collected or processed.	Scope and compliance requirements increase significantly.	High
4	Hosting & tooling: a standard web stack environment is available (Docker-friendly).	Deployment effort increases; environment-specific workarounds needed.	Medium
5	Review access: a clinical consultant can provide limited content review.	Reduced content quality assurance; higher risk of inaccuracies.	Low
6	Time allocation: consistent individual effort over the project window.	Deliverables may be reduced or postponed.	Medium

Constraints

Limitations that affect the project:

Constraint Type	Description	Mitigation
Time	Timeline: ~4 months total.	Prioritize MVP features; timebox analytics and content expansion.
Resources	Team: single developer covering FE/BE/data/content.	Use proven libraries/frameworks; keep architecture simple and testable.
Technology	Technology stack fixed: React, Node.js (Express), Python, PostgreSQL, Docker.	Stick to stack defaults; avoid experimental tooling or unnecessary rewrites.
Budget	No paid tools/datasets; rely on open-source and open data.	Use official open datasets; avoid paid APIs and subscriptions.

Constraint Type	Description	Mitigation
External/Compliance	Respect dataset licenses; GDPR-aware cookie/analytics setup if tracking is added.	Keep tracking optional/minimal; document licenses and citations.
Academic	Deliverables must meet thesis format and review milestones.	Maintain documentation as part of development; schedule review checkpoints.

Dependencies

Dependency	Type	Owner	Status	Why it matters	Management
Official datasets (WHO/EU/national)	External	WHO/Eurostat/CDC/national sources		Indicators and evidence inputs	Track sources & licenses; mirror sample data; record "last updated".
Python ETL toolchain (pandas, etc.)	Technical	Python ecosys-tem		Cleaning/aggregation and SVG export	Pin versions; keep a small test dataset; document script entry points.
PostgreSQL	External	PostgreSQL commu-nity		Persistent disease content storage	Provision dev DB; migra-tions; backup via Docker volume.

Dependency	Type	Owner	Status	Why it matters	Management
Node.js + Express	Technical	Node/Express maintainers		API delivery for the frontend	Lock Node version; keep OpenAPI contract in sync with controllers.
React + Vite	Technical	Meta / Vite community		UI rendering, navigation, build pipeline	Pin dependencies; basic E2E smoke; accessibility checks on key views.
React Router	Technical	React Router maintainers		Client-side routing	Keep route constants centralized; smoke test navigation paths.
Redux Toolkit + React Redux	Technical	Redux maintainers		Shared client state (data fetching/cache)	Keep slices/thunks scoped by feature; test reducers/thunks; avoid unnecessary global state.

Dependency	Type	Owner	Status	Why it matters	Management
Axios	Technical	Axios maintainers		HTTP client for REST API calls	Centralize base URL/interceptors; handle timeouts and errors consistently.
Sentry (frontend)	External	Sentry		Error tracking and performance monitoring	Optional via `SENTRY_DSN`; disable in dev if not configured.
Prisma (ORM) + @prisma/client	Technical	Prisma		Type-safe DB access and schema migrations	Version pinning; run `generate` after schema changes; keep migrations in repo.
Redis (via ioredis)	External	Redis community		Cache for read-heavy endpoints	TTL-based caching; define key conventions; provide safe fallback to DB on cache miss.

Dependency	Type	Owner	Status	Why it matters	Management
OpenAPI docs (express-jsdoc-swagger)	Technical	Library maintainers		API contract documentation	Keep docs close to code; verify `/api-docs` after changes.
Inversify (DI)	Technical	Library maintainers		Backend dependency injection	Keep container bindings consistent; avoid circular dependencies.
Logging (winston, morgan)	Technical	Library maintainers		Operational visibility and debugging	Use `LOG_LEVEL`; structured logs; reduce verbosity in production.
Testing (Jest, Supertest, Playwright)	Technical	OSS maintainers		Regression protection across FE/BE	Run focused unit tests; keep one E2E smoke path for critical pages.
Docker runtime (local/dev) + Docker Compose	External	Docker		Reproducible environment	One `docker-compose.yml`; healthchecks; basic resource limits.

Dependency	Type	Owner	Status	Why it matters	Management
SVG asset hosting (repo)	Technical	Project repo		Serving analytics images deterministically	Folder convention; cache headers; fallback UI if asset missing.
Academic timeline (committee dates)	Organizational	University/committee		Milestone gates for delivery	Backward plan from defense; buffer weeks; freeze scope near deadlines.
Browser support	External	Browser vendors	—	Reader accessibility	Define baseline (e.g., last 2 Chrome/Edge/Firefox + iOS/Android); test key pages.

Stakeholders & Users

Stakeholders Analysis

Stakeholder	Responsibility	Expertise & Task Understanding	Influence	Engagement Strategy
Product Owner & Engineer	Deliver the product: FE/BE, DB, ETL/analytics, documentation	Full-stack, basic clinical/data literacy	High	Weekly plan, task tracker, milestone demos
Academic Supervisor / Committee	Methodology, compliance with requirements, evaluation	Research methodology, thesis requirements	High	Reports and demos at control checkpoints
Clinical Consultant	Review CVD cards, terminology, disclaimer	Cardiology / public health	High (for content)	2–3 review sessions + asynchronous edits
End Users (clinicians, educators, analysts, readers)	Use the library and analytics, provide feedback	High domain expertise / varied technical level	Medium	Short usability tests, on-site feedback form
Data Source Owners (WHO/EU/national)	Dataset access, licensing, attribution	Data stewardship	Medium	License compliance, "References" page, "last updated" note

Target Audience

Persona	Description	Key Needs
Clinician / Medical student	Clinicians, residents, interns, and medical students looking for structured, referenced CVD information with clear symptom/risk-factor separation.	Fast lookup; consistent terminology; references/disclaimer; bilingual UI.

Persona	Description	Key Needs
Educator / Researcher	University educators and academic researchers using the platform for teaching materials, examples, and evidence-linked summaries.	Reliable citations; exportable/quotable summaries; analytics snapshots for lectures; easy navigation.
Analyst / Public health reader	Data analysts and public health readers exploring prevalence/indicators and lightweight breakdowns by age/sex/region.	Simple analytics visuals; source transparency (dataset + last updated); filtering and search; no PII/PHI.

User Personas

Persona 1: Dr. Elena S.

Attribute	Details
Role	Cardiologist / clinical educator
Age	35–45
Tech Savviness	Medium
Goals	Quickly validate terminology and symptom/risk-factor grouping; use referenced summaries for teaching/consultation notes.
Frustrations	Unstructured web sources; inconsistent naming; unclear separation between symptoms vs risk factors; missing citations.
Scenario	During lecture prep or case discussion, searches a disease card, reviews primary/secondary symptoms and prevention, and checks references for credibility.

Persona 2: Alex K.

Attribute	Details
Role	Data analyst (public health / BI)
Age	22–35
Tech Savviness	High

Attribute	Details
Goals	Explore simple prevalence/indicator patterns and per-disease breakdowns; verify dataset provenance and update date.
Frustrations	Data without provenance; outdated datasets; hard-to-reproduce visuals; unclear definitions across sources.
Scenario	Uses search and filters to find a disease, checks the embedded charts, then navigates to the references page to confirm dataset and "last updated" information.

Stakeholder Map

Stakeholders grouped by influence and interest in the project.

High Influence / High Interest

- Product Owner & Engineer: delivery and quality of the full platform.
- Academic Supervisor / Committee: methodology compliance and evaluation.
- Clinical Consultant: medical accuracy of CVD content and disclaimers.

High Influence / Low Interest

- Data Source Owners (WHO/EU/national): license compliance, citation, and correct source representation.

Low Influence / High Interest

- End Users (clinicians, educators, analysts, readers): usability, trust in content, and usefulness of search/filters and analytics.

Low Influence / Low Interest

- General public readers (non-professional): occasional browsing and informational use, without direct influence on scope.

Criterion: Backend Architecture & API Delivery

Architecture Decision Record

Status

Status: Accepted

Date: 2026-01-05

Context

The platform needs a simple, reliable REST API for a read-heavy disease library (pagination, filtering, search) with bilingual content (EN/RU). The backend must be maintainable for a single-developer project and easy to run locally in Docker.

Decision

Implement a TypeScript backend on Node.js with Express using a layered structure (controllers → services → repositories), Prisma for DB access, optional Redis caching, centralized error handling, and OpenAPI docs generated from JSDoc.

Alternatives Considered

Alternative	Pros	Cons	Why Not Chosen
Fastify	High performance; schema-first	Refactor cost	Express already fits the project tooling and patterns
NestJS	Strong structure; DI patterns	More framework overhead	MVP benefits from lighter architecture
FastAPI (Python)	Great for APIs; fits analytics language	Two-stack FE/BE split	Backend already implemented in TS and integrates with FE tooling

Consequences

Positive: predictable request lifecycle, documented endpoints, reproducible local stack.

Negative: DI (Inversify) is not consistently applied across all modules.

Neutral: Redis caching can be expanded only for hot paths.

Implementation Details

Key Implementation Decisions

- Middleware pipeline: JSON → CORS → routes → notFound → error middleware.
- Explicit domain modules (`disease`, `source`) with services and repositories.
- OpenAPI generated close to route/controller definitions.
- Redis cache as an optimization layer (safe fallback to DB on miss).

Project Structure

```
backend/src/
  app/                    # Express app composition
  routes/                 # Route mounting (/health, /diseases, /sources)
  disease/                # Disease domain module (controller/service/repo/cache)
  source/                 # Source domain module
  config/                 # env config keys, swagger, Sentry instrumentation
  middlewares/            # notFound + error middleware
  errors/                 # ApiError + createApiError factory
  cache/                  # Redis client
  utils/                  # Winston logger + Morgan request logger

backend/prisma/
  schema.prisma
  migrations/
  seed.ts
```

Key Implementation Decisions

Decision	Rationale
Express middleware pipeline (`json`, `cors`, routes, notFound, error)	Standard, predictable request lifecycle
Controller/service/repository split	Keeps business logic testable and separate from transport
OpenAPI via JSDoc (`express-jsdoc-swagger`)	Co-locates endpoint docs with implementation
Redis client with env-configured host/port	Enables docker-compose and local dev parity
Graceful shutdown handlers	Avoids abrupt disconnects and resource leaks

Code Examples

Routes and module mounting:

```
// backend/src/routes/index.ts
router.use('/health', healthController.router);
router.use('/diseases', diseaseController.router);
router.use('/sources', sourceController.router);
```

Error handling via typed API errors:

```
// backend/src/disease/disease.controller.ts
if (isNaN(skipValue) || skipValue < 0) {
  throw createApiError.badRequest(msg.SKIP_PARAM_INCORRECT);
}
```

Diagrams

- Container/service wiring is described in `docs/02-technical/deployment.md`.

Requirements Checklist

#	Requirement	Status	Evidence/Notes
1	Express application setup with middleware + routes		`backend/src/app/app.ts`
2	REST endpoints for diseases/sources + health		`backend/src/routes/index.ts`, `backend/src/disease/disease.controller` `backend/src/source/source.controller.t`
3	Pagination/filtering/search for diseases		`backend/src/disease/disease.controller` and service/repo layer
4	Locale-aware responses (en/ru)		`locale` query handling in disease controller/service; DB has translation tables
5	OpenAPI documentation available		JSDoc annotations + `backend/src/config/swagger.ts`
6	Centralized error middleware		`backend/src/middlewares/errorMiddlewar` `backend/src/errors/createApiError.ts`

#	Requirement	Status	Evidence/Notes
7	Redis cache client wired		`backend/src/cache/redisClient.ts`
8	Dependency Injection consistently used		Inversify container exists (`backend/src/container.ts`) but some objects are instantiated manually (routes module)

Known Limitations

Limitation	Impact	Potential Solution
DI is not consistently applied	Harder to swap implementations for testing	Move instantiation into container bindings and resolve controllers/services from DI
Some "no results" cases return 200 + message	Clients must handle two shapes (array vs message)	Consider returning empty arrays with metadata, or a consistent envelope

References

- `backend/package.json`
- `backend/src/app/app.ts`
- `backend/src/routes/index.ts`
- `backend/src/disease/disease.controller.ts`
- `backend/src/cache/redisClient.ts`
- `backend/src/config/instrument.js`

Criterion: Database Design & Data Integrity

Architecture Decision Record

Status

Status: Accepted

Date: 2026-01-05

Context

The platform stores structured disease content (diseases, symptoms, risk factors, sources) with bilingual support (EN/RU) and many-to-many relationships. The schema must enforce integrity while staying easy to evolve during thesis development.

Decision

Use PostgreSQL with Prisma and a normalized schema: core entities (`Disease`, `Symptom`, `RiskFactor`, `Source`), junction tables for M:N relationships with metadata, and translation tables keyed by `locale`.

Alternatives Considered

Alternative	Pros	Cons	Why Not Chosen
Single denormalized diseases table	Very fast to build	Poor integrity; harder filtering; harder bilingual support	Doesn't scale for symptom/risk-factor queries and locale support
Document DB (MongoDB)	Flexible schema	Harder relational queries/integrity; translation duplication	Data is inherently relational and benefits from constraints
Raw SQL access	Full control	Higher maintenance; harder type safety	Prisma provides migrations and type-safe access for TS

Consequences

Positive: - Strong referential integrity (FKs + cascades) - Expressive filtering/search across relationships - Locale support without duplicating core records

Negative: - More tables and joins than a single-table MVP

Neutral: - Translation tables can be extended to additional locales later

Implementation Details

Project Structure

`backend/prisma/`

```
schema.prisma            # Canonical schema
migrations/              # Prisma migration history
seed.ts                  # Seed script
```

Key Implementation Decisions

Decision	Rationale
Junction tables for M:N	Enables per-link metadata (priority/typicality, direction)
Translation tables	Locale-aware fields without duplicating base entity IDs
Enum types	Prevent invalid values and simplify API mapping
Uniques + indexes	Enforce data integrity and speed filtering

Code Examples

Locale enum + translation uniqueness:

```
enum Locale { en ru }

model DiseaseTranslation {
  diseaseId Int
  locale Locale
  @@unique([diseaseId, locale])
}
```

M:N with metadata:

```
model DiseaseSymptom {
  diseaseId Int
  symptomId Int
  priority SymptomPriority
  typicality Typicality
  @@unique([diseaseId, symptomId])
}
```

Diagrams

- The schema is also represented in the project's architecture/ERD diagrams under `docs/assets/diagrams/`.

Requirements Checklist

#	Requirement	Status	Evidence/Notes
1	Relational DB suitable for structured content		`docker-compose.yml` uses `postgres:16`
2	Type-safe ORM layer with migrations		`backend/prisma/schema.prisma`, Prisma migrations
3	Supports bilingual (EN/RU) content		Translation tables + `Locale` enum
4	Supports M:N disease symptom with metadata		`DiseaseSymptom` (priority/typicality)
5	Supports M:N disease risk factor with direction		`DiseaseRiskFactor` (`RiskDirection`)
6	Integrity constraints and indexes		`@@unique`, `@@index` in schema
7	Sources stored for attribution		`Source` model present
8	Seed process available		`backend/prisma/seed.ts` and `npm run seed`

Known Limitations

Limitation	Impact	Potential Solution
`Source` is not linked to diseases in schema	Harder to enforce per-disease references at DB level	Add relation table (e.g., `DiseaseSource`) if needed
Search requirements may need DB-level full-text indexes	Text search may be slower at scale	Add PostgreSQL `tsvector` + GIN indexes if needed

References

- `backend/prisma/schema.prisma`
- `backend/prisma/migrations/`
- `backend/prisma/seed.ts`
- `docker-compose.yml`

Deployment & DevOps

Original documentation: https://docs.google.com/document/d/1yJLejGVrdJQ4Gq0f72P750_r9pzv-XMS-WSrUH_jQEI/edit?usp=sharing

Overview

The platform runs as three services via Docker Compose: API (Node/Express), PostgreSQL, and Redis. This provides a reproducible local environment and a production-like deployment model.

Diagram

Deployment diagram is stored in `docs/assets/diagrams/deployment_architecture_diagram.jpg`.

How to Run (Local)

1. Create a `.env` file (or set environment variables) for the backend.
2. Run `docker compose up --build`.
3. Verify health: `GET /api/health` returns 204.

Environment Variables

Variable	Default	Required	Description	Example
`PORT`	`4000`	Yes	The port on which the Express API server listens. Used by the API container and referenced in health checks.	`4000`

Variable	Default	Required	Description	Example
NODE_ENV	development	No	Runtime environment mode. Set to production for production builds. Affects logging verbosity, error detail exposure, and certain middleware behavior.	production
DATABASE_URL	-	Yes	Full PostgreSQL connection string used by Prisma. Format: postgresql://USER:PASSWORD@HOST:PORT/DATABASE.	postgresql://cvd_user:password@loca
POSTGRES_USER	-	Yes	PostgreSQL username. Used by the db container in docker-compose.yml.	postgres
POSTGRES_PASSWORD	-	Yes	PostgreSQL password. Used by the db container.	password
POSTGRES_DB	-	Yes	PostgreSQL database name. Used by the db container.	cvd_db

Variable	Default	Required	Description	Example
`REDIS_HOST`	`localhost`	Yes	Hostname or IP of the Redis server. Set to `redis` when running via Docker Compose.	`redis`
`REDIS_PORT`	`6379`	Yes	Port on which Redis is listening.	`6379`
`SENTRY_DSN`	-	No	Sentry project DSN for error tracking and performance monitoring. If not provided, Sentry integration is disabled.	`https://...`
`CORS_ORIGIN`	`*`	No	Allowed origin(s) for CORS requests. Set to your frontend URL in production (e.g., `https://your-frontend.com`).	`http://localhost:5173`

Variable	Default	Required	Description	Example
`API_URL`	-	No	Base URL of the backend REST API, used by the frontend and tooling to build requests (e.g. `http://localhost:4000/api`). Typically matches the external address of the `/api` prefix in development and production.	`http://localhost:4000/api`
`CLIENT_URL`	-	No	Base URL of the frontend application that consumes this API (e.g. `http://localhost:5173`). Can be used for redirects, links in error messages, or security checks when needed.	`http://localhost:5173`

Variable	Default	Required	Description	Example
`LOG_LEVEL`	`info`	No	Minimal log level for the application logger (e.g. `info`, `debug`, `warn`, `error`). This controls verbosity of JSON logs and is useful to reduce noise in production while keeping more detailed output in development.	`debug`

Notes

- Health checks are configured for API/DB/Redis in `docker-compose.yml`.
- Persistent data is stored in Docker volumes for PostgreSQL and Redis.

Criterion: Frontend Architecture & UX Delivery

Architecture Decision Record

Status

Status: Accepted

Date: 2026-01-05

Context

The project needs a responsive UI to browse/read disease content with search/filters and EN/RU support. The frontend should be fast to iterate and

testable in a single-developer timeline.

Decision

Build a React + TypeScript SPA using Vite. Use React Router for navigation, Redux Toolkit for shared data flow, an Error Boundary + Sentry for crash handling, and SCSS modules for themeable styling.

Alternatives Considered

Alternative	Pros	Cons	Why Not Chosen
Next.js (SSR/ISR)	Better SEO defaults; SSR capabilities	More framework complexity; additional server concerns	MVP is a read-focused library where SSR is not required; faster iteration preferred
No global store (local state only)	Less boilerplate for small apps	Harder cross-page caching and shared data	Shared data (diseases/sources/risk factors/symptoms) benefits from a predictable state layer
UI framework (e.g., MUI)	Faster component assembly	Design constraints, heavier runtime	Project already uses SCSS modules and shared UI components

Consequences

Positive: - Fast dev experience and builds via Vite - Predictable state/data flow via Redux Toolkit - Centralized error capture (Sentry + Error Boundary) - Testability (Jest/RTL + Playwright; Storybook)

Negative: - SPA navigation requires explicit handling for SEO basics - Global store adds some complexity for small pages

Neutral: - Some routes/pages are scaffolded but not fully wired yet (see checklist)

Implementation Details

Project Structure

```
frontend/src/
  app/
     App.tsx                          # BrowserRouter + providers
     layouts/MainLayout/              # Header + <Outlet /> shell
     providers/
         ErrorBoundary/               # React Error Boundary
         Router/                      # Route config + lazy pages
         StoreProvider/               # Redux store provider
  pages/                               # Route-level pages (Error/NotFound implemented)
  shared/
     api/                             # API client + slices/thunks
     context/                         # Theme context
     hooks/                           # useTheme and helpers
     ui/                              # Shared UI components (Header, Loader, etc.)
  assets/                              # SVG assets
```

Key Implementation Decisions

Decision	Rationale
BrowserRouter-based SPA	Simple routing for a content library
Lazy-loading for route pages	Reduce initial bundle; faster first paint
Global layout (MainLayout)	Consistent header + content; shared data prefetch
Redux Toolkit store	Shared async data fetching and caching across views
Sentry + ErrorBoundary	Centralized error capture and user-friendly fallback

Code Examples

Provider composition (routing + error boundary + store):

```
// frontend/src/app/App.tsx
<BrowserRouter>
  <ErrorBoundary>
    <StoreProvider>
      <Router />
    </StoreProvider>
  </ErrorBoundary>
</BrowserRouter>
```

Redux store assembly (RTK + RTK Query middleware):

```
// frontend/src/app/providers/StoreProvider/config/store.ts
const rootReducer = combineReducers({
  [api.reducerPath]: api.reducer,
  diseases: diseasesReducer,
  sources: sourcesReducer,
});
```

Diagrams

- High-level deployment/service interaction is documented in the technical deployment chapter.

Requirements Checklist

#	Requirement	Status	Evidence/Notes
1	SPA entrypoint and routing foundation		`frontend/src/app/App.tsx`, `frontend/src/app/providers/Router/ui/R`
2	Centralized layout shell		`frontend/src/app/layouts/MainLayout/Ma`
3	Shared store for API data		`frontend/src/app/providers/StoreProvic`
4	Error handling (user + monitoring)		`frontend/src/app/providers/ErrorBounda` `frontend/src/main.tsx`
5	Theme context available		`frontend/src/shared/hooks/useTheme.ts`
6	Main library pages (Main/Sources/Disease) wired to router		Routes exist but are currently commented in `Router.tsx`
7	Research cards/gallery route implemented		Route constant exists (`/research`), but no page is mounted in router

#	Requirement	Status	Evidence/Notes
8	Automated testing for critical flows		Jest + Playwright configured; tests present under `frontend/tests/e2e/` and `frontend/src/**/__tests__`

Known Limitations

Limitation	Impact	Potential Solution
Core content pages not mounted in router	Most user stories remain inaccessible in UI	Uncomment/implement `MainPage`, `SourcePage`, `DiseasePage` and add `ResearchPage`
SPA SEO is limited by default	Discoverability may be reduced	Add sitemap/meta tags, and consider SSR only if needed

References

- `frontend/package.json`
- `frontend/src/app/App.tsx`
- `frontend/src/app/providers/Router/ui/Router.tsx`
- `frontend/src/app/providers/StoreProvider/config/store.ts`
- `frontend/src/main.tsx`

Criterion: Observability (Logging + Error Monitoring)

Architecture Decision Record

Status

Status: Accepted

Date: 2026-01-05

Context

The MVP needs enough observability to diagnose API issues and client crashes during demos: request logging, structured app logs, and error reporting. The

setup must remain lightweight.

Decision

Use Sentry for error monitoring (frontend + backend). Use Winston for backend application logs and Morgan for HTTP request logs. Add a frontend Error Boundary to prevent blank screens and capture exceptions.

Alternatives Considered

Alternative	Pros	Cons	Why Not Chosen
Full APM suite	Rich diagnostics	Cost + setup	Too heavy for MVP
Console-only logs	Minimal work	Hard triage	No aggregation/structure
No client monitoring	Simpler	Silent UI failures	Poor UX and no visibility

Consequences

Positive: faster debugging and clearer demo reliability.

Negative: Sentry must be configured to avoid sending sensitive data.

Neutral: sampling/verbosity can be tuned later.

Implementation Details

Key Implementation Decisions

- Skip noisy endpoints (e.g., health checks) in request logging.
- Keep monitoring optional via environment variables.

Project Structure

```
frontend/src/main.tsx
frontend/src/app/providers/ErrorBoundary/
backend/src/config/instrument.js
backend/src/utils/logger.ts
backend/src/utils/requestLogger.ts
```

Requirements Checklist

#	Requirement	Status	Evidence/Notes
1	Frontend crash fallback		ErrorBoundary
2	Frontend error reporting		Sentry init

#	Requirement	Status	Evidence/Notes
3	Backend error reporting		Sentry init + handlers
4	Backend structured logs		Winston logger
5	HTTP request logs		Morgan middleware

Known Limitations

Limitation	Impact	Potential Solution
Sentry may capture extra context	Privacy risk	Keep PII/PHI out of the app and disable default PII collection
Monitoring disabled if DSN missing	Less visibility	Require DSN for staging/prod demos

Criterion: Research Pipeline & Analytics Artifacts

Architecture Decision Record

Status

Status: Accepted

Date: 2026-01-05

Context

The thesis requires dataset-based insights, but the product scope avoids real-time BI and heavy analytics infrastructure. The pipeline must be reproducible and reviewable.

Decision

Keep research/ETL in Python under `analysis/` with datasets committed for reproducibility. Generate plots/findings as static artifacts (SVG/PNG) intended to be embedded in the web UI.

Alternatives Considered

Alternative	Pros	Cons	Why Not Chosen
Power BI/Fabric	Rich dashboards	Vendor + deployment overhead	Out of scope for MVP
Frontend-only charts	No separate pipeline	Hard reproducibility	Analysis must be verifiable
Warehouse + scheduled jobs	Scales well	Too heavy	Not needed for read-only library

Consequences

Positive: transparent, in-repo workflow; low runtime complexity.

Negative: visuals are mostly static.

Neutral: pipeline can be formalized later (pinned env + standardized export).

Implementation Details

Key Implementation Decisions

- Separate research code from API runtime.
- Prefer deterministic exports over live analytics services.

Project Structure

```
analysis/
    datasets/
    code_solution/
```

Requirements Checklist

#	Requirement	Status	Evidence/Notes
1	Research code in repo		`analysis/code_solution/*`
2	Datasets included		`analysis/datasets/*`
3	Standard Python toolchain		pandas/numpy/matplotlib usage
4	Exportable artifacts		Plots exist; export convention can be standardized

#	Requirement	Status	Evidence/Notes
5	Not coupled to runtime		No API dependency on Python pipeline

Known Limitations

Limitation	Impact	Potential Solution
Dependencies not pinned	Repro varies by machine	Add an `analysis/requirements.txt`
No single outputs folder	Harder embedding	Standardize `analysis/outputs/` exports

Criterion: Testing & Quality Assurance

Architecture Decision Record

Status

Status: Accepted

Date: 2026-01-05

Context

The project needs basic automated tests to reduce regressions while iterating quickly across a full-stack codebase (frontend + backend). Tests must be lightweight and runnable locally.

Decision

Use Jest for unit tests in both backend and frontend, with focused tests around error handling/config/utilities. Use Playwright for end-to-end smoke coverage in the frontend where applicable.

Alternatives Considered

Alternative	Pros	Cons	Why Not Chosen
No tests	Fastest to write features	Regressions likely	Not suitable for thesis-quality delivery

Alternative	Pros	Cons	Why Not Chosen
Only E2E tests	Real user flows	Slower, brittle	Too heavy for quick iteration
Only unit tests	Fast, stable	Miss integration issues	Add E2E later for key flows

Consequences

Positive: faster refactors, confidence in core helpers.

Negative: some UI routes are not fully wired, limiting E2E value.

Neutral: test coverage can be expanded when routes stabilize.

Implementation Details

Key Implementation Decisions

- Keep unit tests small and deterministic (no external network calls).
- Prefer testing error factories/config parsing and domain services.

Project Structure

```
backend/tests/
frontend/tests/
```

Requirements Checklist

#	Requirement	Status	Evidence/Notes
1	Backend unit tests exist		`backend/tests/*`
2	Frontend unit test setup exists		Jest/RTL config + setup
3	E2E test harness exists		Playwright config + tests folder
4	CI-friendly commands available		Depends on project scripts

Known Limitations

Limitation	Impact	Potential Solution
Limited coverage of user stories	Regressions possible in unwired routes	Add tests as routes/features are completed

Technology Stack

Stack Overview

Layer	Technology	Version	Justification
Frontend	React + TypeScript + Vite	React 19.1.1, TS ~5.8.3, Vite 6.3.5	Fast dev/build for SPA, simple routing, easy theming and component iteration.
UI Framework	SCSS Modules + Sass	Sass 1.89.2	Lightweight styling approach aligned with the MVP; supports theming (Dark/Light) without adopting a heavy UI framework.
Backend	Node.js + TypeScript	Node.js 20, TS 5.9.3	Familiar ecosystem, fast iteration, strong support for REST APIs and tooling.
Framework	Express	5.1.0	Lightweight HTTP framework; easy JSON APIs and static delivery; pairs well with OpenAPI tooling.
Database	PostgreSQL	16 (Docker image: `postgres:16`)	Relational integrity for a structured knowledge base (diseases, symptoms, risk factors, translations).
ORM	Prisma	6.19.0	Type-safe DB access, migrations, and schema evolution; good developer ergonomics for TS.
Cache	Redis (+ ioredis client)	Redis 7 (`redis:7-alpine`), ioredis 5.8.2	Simple caching layer for read-heavy endpoints and deterministic assets.
Message Queue	Not used (MVP)	-	Not required for read-only library MVP; can be added if async ingestion/jobs grow.
Analytics	Python ETL + pre-rendered SVG exports	Python 3.x (analysis scripts)	Deterministic, cacheable visual outputs; avoids runtime BI dependencies.

Layer	Technology	Version	Justification
Deployment	Docker + Docker Compose	-	Reproducible local/prod-like environment; simple multi-service orchestration (API + DB + Redis).

Key Technology Decisions

Decision 1: Frontend — React + Vite

Context: Build a responsive SPA for a read-only disease library (cards + search + filters + references/research views) with fast iteration.

Decision: React + Vite.

Rationale: - Fast dev server and build pipeline for frequent UI changes. - Works well for SPA navigation and theming (Dark/Light). - Strong ecosystem for UI components, routing, and testing.

Trade-offs: - Pros: fast iteration, simple SPA architecture, flexible component model. - Cons: no SSR by default; SEO requires basic SPA-friendly practices.

Alternative considered: Next.js (SSR/ISR, stronger SEO) — rejected for MVP due to added complexity not needed for a largely article/library UI.

Decision 2: Backend — Node.js + Express

Context: Provide a simple, stable REST API for diseases/symptoms/risk factors/sources with OpenAPI docs and predictable responses.

Decision: Node.js + Express.

Rationale: - Lightweight and familiar for JSON APIs. - Fits well with a React SPA and Docker-based local environment.

Trade-offs: - Pros: fast iteration, minimal boilerplate, strong middleware ecosystem. - Cons: requires separate Python path for analytics scripts (kept decoupled by design).

Alternative considered: Django (Python) — rejected for MVP because it is a heavier stack and slower iteration with an existing JS frontend.

Decision 3: Analytics & Visualization — Python ETL + SVG embeds

Context: Generate simple, trustworthy visualizations without adding complex BI runtime dependencies.

Decision: Python ETL + pre-rendered SVG embeds.

Rationale: - Deterministic outputs suitable for caching and static delivery. - Avoids licensing/vendor lock-in and minimizes runtime complexity. - Keeps analytics development independent from the web runtime.

Alternative considered: Power BI/Fabric or live JS charting only — rejected for licensing/vendor lock-in and increased runtime complexity; interactive JS charts can be added later if needed.

Decision 4: State management — Redux Toolkit (current) vs local state (initial idea)

Context: MVP initially expected mostly local/query state for cards/search/filters.

Decision: Redux Toolkit is present and used for data fetching and shared state in the current codebase (thunks + slices; RTK Query scaffold).

Rationale: - Centralizes API data lifecycle (loading/error/results) for multiple views. - Easier to extend for caching, saved views, and more complex UI flows.

Alternative considered: No Redux initially — acceptable for a very small UI, but the project already includes Redux Toolkit to support growth.

Decision 5: Database modeling — normalized schema (implemented)

Context: The MVP text mentions a single diseases table to ship fast.

Decision: The implemented schema is normalized: `Disease`, `Symptom`, `RiskFactor`, join tables (`DiseaseSymptom`, `DiseaseRiskFactor`), `Source`, and translation tables per locale.

Rationale: - Supports primary/secondary symptom modeling and risk factor direction. - Supports localization (EN/RU) at the data layer. - Improves queryability and data integrity compared to a single denormalized table.

Development Tools

Tool	Purpose	Notes
IDE	VS Code	ESLint/Prettier integration, TypeScript support.
Version Control	Git	MVP workflow; hooks enforce formatting/linting.
Package Manager	npm	Frontend/backend dependency management.

Tool	Purpose	Notes
Linting & Formatting	ESLint + Prettier (+ Stylelint for SCSS)	Auto-fix enabled; lint-staged + Husky pre-commit.
Testing (FE)	Jest + React Testing Library; Playwright (E2E)	Unit/component + end-to-end coverage.
Testing (BE)	Jest + Supertest	Controller/service/repository tests.
Build tooling (BE)	SWC	Fast TypeScript compilation to `build/`.
DB Tooling	Prisma CLI	Migrations, client generation, seeding.
Documentation	OpenAPI (express-jsdoc-swagger), Storybook	API docs at `/api-docs`; component docs via Storybook.
Containers	Docker + Docker Compose	Runs API + PostgreSQL + Redis locally.

External Services & APIs

Service	Purpose	Pricing Model
Sentry	Error monitoring for frontend and backend	Free tier / paid plans
WHO Data	Official datasets for indicators and references	Free (open data; license-dependent)
Eurostat	Official EU statistics (indicators by region)	Free (open data; license-dependent)
CDC WONDER	Official public-health datasets	Free (open data; terms apply)

FAQ & Troubleshooting

FAQ

What is this platform for?

It is a read-only CVD knowledge platform: diseases are presented as standardized entries (symptoms, risk factors, prevention guidance) with attribution to sources. It is intended for education and exploration.

Is it medical advice?

No. It does not replace professional medical advice, diagnosis, or treatment.

Q: What data does the platform store about me?

A: In the MVP, the platform is designed to avoid collecting personal health information (PHI) and personally identifiable information (PII). It focuses on read-only reference content (diseases, symptoms, risk factors, sources) and system telemetry (logs/metrics) for reliability.

Q: What content can I expect to see?

A: You can expect disease reference entries with structured symptom/risk-factor lists and prevention text. You may also see "Research Cards" (view-only research summaries) depending on the current UI wiring.

Q: Which languages are supported?

A: The backend data model supports English and Russian locales (`en`, `ru`). If the UI exposes a language switch, it will typically map to the backend `locale` parameter; otherwise developers can still request a locale through the API.

Account & Access

Q: Do I need an account to use the platform?

A: Not for the current MVP read-only scope. There is no login/registration requirement for browsing content.

Q: Why can't I access the application or some pages?

A: If you are running locally, the most common causes are that the backend/API is not running, the frontend dev server is not running, or the intended routes are not fully wired in the current build. Verify the services are up and check that the frontend points to the correct API base URL.

Features

Q: How do I search diseases?

A: The API supports free-text search (via a `search` query parameter) and filtering (e.g., by symptom or risk factor). In the UI, this is typically exposed as a search input and/or filters; if you are integrating directly, call the diseases endpoint with the relevant query parameters.

Q: Why do I sometimes get "No disease found." but the request is successful?

A: Some list endpoints intentionally return HTTP `200` with a JSON `{ "message": "No disease found." }` (or `{ "message": "No source found." }`) when a query returns no results. This is a "no-results" response, not an error.

Q: What are "primary" vs "secondary" factors?

A: The project's data model separates core/primary symptoms from secondary symptom-related factors to keep disease cards readable and to support more structured filtering and analysis.

Q: Where does the content come from?

A: The platform stores and displays a list of official or curated sources (organizations/datasets) and links out to them. In the backend, these appear as "Sources" with a name and a URL.

Q (Developer): What endpoints are available?

A: The backend exposes read-only endpoints under `/api`, including health checks, diseases, symptoms, risk factors, and sources. See the API reference at appendices/api-reference.md and Swagger at `http://localhost:4000/api-docs` when running locally.

Q (Developer): What ports does the local setup use?

A: Common defaults are:

- Backend API: `http://localhost:4000/api`
- Swagger UI: `http://localhost:4000/api-docs`
- Frontend dev server (Vite): typically `http://localhost:5173`

Your local `.env` / Docker Compose configuration is the source of truth.

Q (Developer): How do I run everything locally?

A: The project supports Docker Compose for a full local environment (API + PostgreSQL + Redis). Alternatively, you can run backend and frontend separately with `npm run dev` in each package.

Q (Developer): How do I reset/reseed the database?

A: From the backend folder you can run migrations and seeding scripts (see appendices/db-schema.md). If your dataset changes a lot during development, prefer a reset workflow to ensure schema + seed data are consistent.

Troubleshooting

Common Issues

Problem	Possible Cause	Solution
Page won't load / blank screen	Frontend dev server not running; JS error; wrong base URL	Start frontend, open browser devtools console, confirm API base URL is correct and reachable
API requests fail with `ERR_CONNECTION_REFUSED`	Backend not running or wrong port	Start backend (or Docker Compose); confirm `http://localhost:4000/api/health` responds
Browser shows CORS error	`CORS_ORIGIN` / `CLIENT_URL` mismatch	Ensure backend CORS origin matches the frontend URL (e.g., `http://localhost:5173`)
"No disease found." even though data should exist	Empty DB; seed not applied; filters too strict	Run migrations + seed; retry with no filters (remove `search`, `symptom`, `riskFactor`, `letter`)
Swagger (`/api-docs`) not loading	Backend down; reverse proxy misconfigured	Confirm backend is running and try `http://localhost:4000/api/health/details`

Problem	Possible Cause	Solution
DB errors on startup (Prisma connection)	`DATABASE_URL` wrong; DB container not healthy	Verify Postgres container health, then check `DATABASE_URL` format and credentials
Redis connection errors	Redis container not running; wrong host in Docker	In Docker Compose, Redis host is typically `redis`; outside Docker it is often `localhost`

Error Messages

Error Code/Message	Meaning	How to Fix
`{ "message": "No disease found." }` (HTTP 200)	Query returned zero results (not an error)	Relax filters/search, confirm DB is seeded
`{ "success": false, "message": "Bad Request", ... }` (HTTP 400)	Invalid query/body parameters	Check parameter types/ranges (e.g., `skip >= 0`, `1 <= take <= 100`)
`{ "success": false, "message": "Resource not found: ..." }` (HTTP 404)	Route does not exist	Confirm the path starts with `/api/...` and matches the API reference
`{ "success": false, "message": "Internal Server Error" }` (HTTP 500)	Unhandled backend error	Check backend logs; in development you may see a stack trace in the response

Browser-Specific Issues

Browser	Known Issue	Workaround
Chrome	Aggressive caching during local dev	Hard refresh (`Ctrl+F5`) and disable cache in devtools while debugging
Firefox	CORS errors are sometimes shown differently	Check the Network tab for the actual failing request and response headers

Browser	Known Issue	Workaround
Safari	Mixed-content restrictions when frontend is `https` and API is `http`	Use consistent schemes locally (usually both `http`)
Edge	Same behavior as Chrome (Chromium)	Apply the same cache + devtools steps as Chrome

Getting Help

Self-Service Resources

- Documentation
- API Reference
- Swagger/OpenAPI: `http://localhost:4000/api-docs` (when running locally)
- Repository: https://github.com/Doshirac/cvd-platform

Contact Support

Channel	Response Time	Best For
GitHub Issues	Best-effort	Bug reports, feature requests, documentation problems

Reporting Bugs

When reporting a bug, please include:

1. **Steps to reproduce** - What actions lead to the issue?
2. **Expected behavior** - What should happen?
3. **Actual behavior** - What actually happens?
4. **Screenshots** - If applicable
5. **Browser/Device info** - Browser name, version, OS

Submit bug reports at: [Issue tracker URL or email]

Suggested issue tracker: - https://github.com/Doshirac/cvd-platform/issues

Feature Walkthrough

This guide explains the core user flows of the CVD Platform.

Notes: - Screenshot links are placeholders — you can add the image files later. - Some routes/pages are still being wired in the current frontend router. If you

don't see a page described below, check the Not Found page and the project's current UI status. - Some UI interactions in this chapter are marked as "if enabled in your build", because different project revisions may have unfinished wiring. - Original UI/UX Documentation

Feature 1: Navigation, Theme & Language

Overview

Use the header to navigate between the main sections of the platform and to adjust UI preferences (theme, language). This improves usability for both desktop and mobile users.

How to Use

Header navigation Image

Step 1: Use the header navigation - Click **Home**, **Sources**, or **Research** in the top navigation. - On mobile screens, open the menu via the burger button.

Page Navigation Example on the mobile - Here you can find the schema

Page Navigation Example on the desktop - Here you can find the schema

Step 2: Switch the theme (Light/Dark) - Click the theme toggle icon in the header. - The chosen theme is saved and restored on the next visit.

Theme Changing Example Visualization on the mobile - Here you can find the schema

Step 3: Open the language selector - Click the globe icon in the header and choose a language option. - Depending on the current build, the selector may be UI-only (visual selection) or may also affect content loading.

Language Switching Example Visualization on the mobile - Here you can find the schema

Expected Result: You can navigate between sections, and your UI preferences (especially theme) persist.

Tips

- If a page route isn't available yet, you may land on a **Not Found** page even if the header contains the link.
- If the theme looks "stuck", hard-refresh the page and ensure your browser allows localStorage.

Feature 2: Browse the Disease Library (Home)

Overview

Browse a list of cardiovascular diseases presented as standardized cards. This is the main entry point for discovery.

How to Use

Disease library Image

Step 1: Open the Home page - Use the **Home** link in the header.

Step 2: Review the list of diseases - Scroll the list and open a disease that matches your interest. - If pagination controls are present, use them to navigate through pages.

2.1 Disease Page Navigation (if enabled in your build)

Clicking a disease card navigates to a disease details page containing all information about the selected disease.

Disease Page Navigation Visualization - Here you can find the schema

Step 3: Continue exploring - Return to the list to compare several diseases.

Expected Result: A disease list is displayed with clear loading/empty states and navigation to deeper details.

Feature 3: Search & Filter Diseases

Overview

Use search and filters to quickly find diseases relevant to symptoms, risk factors, or keywords. This supports faster discovery than manual browsing.

How to Use

Search and filters Image

3.1 Searching for a disease by name or code

Use the Search Bar to find a disease (or group of diseases) by name, medical code, symptom, or risk factor.

Example of disease search by Search Bar Visualization - Here you can find the schema

Step 1: Search by keywords - Enter a keyword related to a disease name, symptom, risk factor, or code. - Review the results list.

3.2 Filtering diseases by filters and risk factors

Use the Filter Panel to narrow the disease list by selecting symptoms/risk factors (including primary/secondary grouping where available).

Filter panel expanded/collapsed Visualization - Here you can find the schema

Step 2: Narrow results using filters - Apply a **Symptom** filter (by term or code) and/or a **Risk Factor** filter (by name or code). - Adjust filters until the list matches your intent.

Mobile filtering flow On mobile, filters may be located behind a filter icon.

Open filtering panel on mobile Visualization - Here you can find the schema

Expected Result: The results list updates and shows either matching diseases or a clear "no results" state.

3.3 Alphabet Filtering (if enabled in your build)

Some builds allow filtering the disease list by the first letter (A–Z). Selecting a letter filters the list to diseases starting with that letter. Selecting the same letter again (or using "clear") resets the filter.

Finding Diseases that start from A character Visualization - Here you can find the schema

If no diseases match the chosen letter (or combined filters), the UI should show a clear "no results" scenario.

No Disease Found Visualization - Here you can find the schema

3.4 Loading more items on the list (if enabled in your build)

Some builds provide a **See more** button to load additional cards without leaving the page.

See more button Visualization - Here you can find the schema

Expected behavior: - Clicking **See more** loads 5 additional cards into the list. - The same interaction pattern may be used on the Sources and Research pages.

Tips

- If you see a successful response but no items (for example, "No disease found."), it usually means the filters/search are too strict or the database is not seeded.
- If you are integrating directly via API, the diseases endpoint supports `search`, `symptom`, `riskFactor`, pagination (`skip`/`take`), and locale.

Feature 4: Sources & Research Cards

Overview

View the list of sources used for attribution (datasets/organizations), and browse research-oriented, view-only "Research Cards" when available. This supports transparency and traceability.

How to Use

Step 1: Open Sources - Navigate to **Sources** from the header. - Use search (if available) to find a specific organization.

Step 2: Open a source link - Click the **View Resource** button (or source link) to open the external resource in a new tab and verify provenance.

View Source Visualization - Here you can find the schema

Step 3: Open Research - Navigate to **Research** from the header. - Browse "Research Cards" for short summaries and supporting visuals (if the page is enabled in your build).

Research cards Visualization - Here you can find them

Expected Result: You can see references/sources for transparency and (optionally) read research summaries on the website.

4.1 Research Card Navigation (if enabled in your build)

Some builds allow opening an individual research card to view its full content.

Research Card Opening Visualization - Here you can find the schema

To close the opened research card and return to the list, use the cross icon or a **Close** button.

Closing Research Card Visualization - Here you can find the schema

Feature 5: Back to Diseases List Feature (if enabled in your build)

Instead of using the header navigation or mobile burger menu, some builds provide a **Back to Diseases** button for quicker return to the main disease list.

Back to Diseases List Visualization - Here you can find the schema

Feature 6: Scroll-to-Top Button (if enabled in your build)

6.1 Scrolling to the Top Feature

On long pages, a Scroll-to-Top button can be used to return to the top quickly.

Scroll-to-Top Button Visualization - Here you can find the schema

Feature 7: Hints / Tooltip Badges (if enabled in your build)

7.1 Hints Text Feature

Hovering (desktop) or long-pressing (mobile, if supported) a badge with a medical code (e.g., "SOB") can show a tooltip with the full symptom/risk name.

Tooltip Badge Visualization - Here you can find the schema

Keyboard Shortcuts

Shortcut	Action
`Ctrl/Cmd + R`	Refresh the current page
`Ctrl/Cmd + L`	Focus the browser address bar
`Alt + ←` / `Cmd + [`	Navigate back
`Ctrl/Cmd + F`	Find text on the current page
`Esc`	Close a dropdown/menu (when applicable)

Feature Comparison

Feature	MVP (Diploma scope)	Post-MVP (Possible extensions)
Browse/search/filter disease library		
Disease details (standardized card sections)	UI wiring may be in progress	
Sources & attribution		(per-disease linking could be expanded)
Research Cards (view-only research summaries)	May be disabled depending on current build	(more research cards + better presentation)

API Reference

Overview

Base URL: `http://localhost:4000/api`

Endpoints

Check API health

GET /health Verify that the API service is running.

Request:

```
GET /api/health
```

Response:

```
{}
```

Status Code	Description
204	No Content

GET /health/details Get detailed health information.

Request:

```
GET /api/health/details
```

Response:

```
{
  "status": "text",
  "timestamp": "text",
  "uptime": 1,
  "environment": "text",
  "system": {
    "platform": "text",
    "nodeVersion": "text",
    "memory": {
      "total": "text",
      "free": "text",
      "usage": "text"
    },
    "cpu": "text",
    "cores": 1
  }
}
```

Status Code	Description
200	Success

Retrieve the diseases

GET /diseases Retrieve diseases with pagination and filtering.

Query Parameters:

Parameter	Type	Required	Description
skip	integer	No	Number of records to skip (default: 0, min: 0)
take	integer	No	Number of records to take (default: 6, min: 1, max: 100)
symptom	string	No	Filter by symptom (term or SNOMED code)
riskFactor	string	No	Filter by risk factor (name or code)
search	string	No	Free text search across diseases, symptoms, risk factors
locale	string	No	Language locale (“en” or “ru”, default: “en”)

Response (200):

Returns either:

- an array of Disease objects (when results exist), or
- an object with a message (when no diseases are found).

Disease object:

Field	Type	Description
id	number	Disease identifier
code	string	ICD-10 code
name	string	Disease name
description	string	Detailed description
prevention	string	Prevention recommendations
symptoms	string[]	List of symptoms
risks	string[]	List of risk factors

Request:

```
GET /api/diseases?skip=0&take=6&search=coronary&locale=en
```

Response (example):


```
[
  {
    "id": 1,
    "code": "CAD",
    "name": "Coronary Artery Disease",
    "description": "A condition where the coronary arteries become narrowed or blocked",
    "prevention": "Regular exercise, healthy diet, quit smoking",
    "symptoms": [
      "chest pain",
      "shortness of breath",
      "fatigue"
    ],
    "risks": [
      "smoking",
      "high cholesterol",
      "diabetes"
    ]
  }
]
```


Response (example — no diseases found):


```
{
  "message": "No disease found."
}
```


Status Code	Description
200	List of diseases
400	Invalid query parameters
500	Internal server error

GET /diseases/risk-factors Get all risk factors with codes.

Query Parameters:

Parameter	Type	Required	Description
locale	string	No	Language locale (“en” or “ru”, default: “en”)

Response (200):

Returns an array of Risk Factor objects.

Risk Factor object:

Field	Type	Description
code	string	Risk factor code
name	string	Risk factor name
definition	string	Risk factor definition

Request:

```
GET /api/diseases/risk-factors?locale=en
```

Response (example):

```
[
  {
    "code": "SM",
    "name": "Smoking",
    "definition": "Tobacco use including cigarettes, cigars, and pipes"
  },
  {
    "code": "HG",
    "name": "High cholesterol",
    "definition": "Elevated levels of cholesterol in the blood"
  }
]
```

Status Code	Description
200	List of risk factors
500	Internal server error

GET /diseases/by-letter Retrieve diseases by initial letter (locale-aware).

Query Parameters:

Parameter	Type	Required	Description
letter	string	No	Initial letter to filter by (single character)
skip	integer	No	Number of records to skip (default: 0)
take	integer	No	Number of records to take (default: 6)

Parameter	Type	Required	Description
locale	string	No	Language locale (“en” or “ru”, default: “en”)

Request:

```
GET /api/diseases/by-letter?letter=C&skip=0&take=6&locale=en
```

Response (200):

Returns either:

- an array of Disease objects (when results exist), or
- an object with a message (when no diseases are found).

Response (example — no diseases found):

```
{
  "message": "No disease found."
}
```

Status Code	Description
200	List of diseases
400	Invalid query parameters
500	Internal server error

Sources

GET /sources Get sources with pagination and optional search.

Query Parameters:

Parameter	Type	Required	Description
skip	integer	No	Number of records to skip (default: 0, min: 0)
take	integer	No	Number of records to take (default: 6, min: 1, max: 100)
search	string	No	Search by source / organization name

Response (200):

Returns either:

- an array of Source DTO objects (when results exist), or
- an object with a message (when no sources are found).

Source DTO object:

Field	Type	Description
id	number	Source identifier
name	string	Source / organization name
link	string	URL to the data source

Request:

```
GET /api/sources?skip=0&take=6&search=world
```

Response (example):

```
[
  {
    "id": 1,
    "name": "World Health Organization",
    "link": "https://www.who.int"
  },
  {
    "id": 2,
    "name": "American Heart Association",
    "link": "https://www.heart.org"
  }
]
```

Response (example — no sources found):

```
{
  "message": "No source found."
}
```

Response (400 example — invalid skip):

```
{
  "success": false,
  "message": "Parameter 'skip' must be a non-negative integer"
}
```

Response (400 example — invalid take):

```
{
  "success": false,
  "message": "Parameter 'take' must be between 1 and 100"
}
```

Status Code	Description
200	List of sources
400	Invalid query parameters
500	Internal server error

Error Responses

All errors are returned in a consistent JSON format.

Error response format (from the global error middleware):

```
{
  "success": false,
  "message": "Human-readable error message",
  "errors": ["Optional validation errors"],
  "stack": "Optional stack trace (development only)"
}
```

Status codes and default messages (from `createApiError`):

Status Code	Default message	When it happens
400	Bad Request	Invalid query/body parameters, failed validation
401	Unauthorized	Missing/invalid authentication (if enabled for the route)
403	Forbidden	Authenticated but not allowed (if enabled for the route)
404	Not Found	Resource not found (can also be returned by the not-found middleware as `Resource not found: <url>`)
405	Method Not Allowed	HTTP method not supported for the endpoint
409	Conflict	Conflict with existing state (e.g., duplicate resource)
500	Internal Server Error	Unhandled server error

Swagger/OpenAPI

Full API documentation available at: `https://app.gitbook.com/invite/YJjvuHTqbLlvmjZEzQci/J36lOoZ1h`

Database Schema

Overview

Attribute	Value
Database	PostgreSQL
ORM	Prisma
Schema source	backend/prisma/schema.prisma
Documentation link	https://docs.google.com/document/d/1FeM152YZaFifvUPv-9Nsqit8NmtA2ZMBfu1xeoPYsN0/edit?usp=sharing

Entity Relationship Diagram

Diagram - Here you can find the diagram

Enums

These enums are defined in the database (PostgreSQL enum types) and used by multiple tables.

Enum	Values	Used in
Locale	`en`, `ru`	`disease_translation`, `symptom_translation`, `risk_factor_translation`
SymptomCategory	`sign`, `symptom`	`Symptom.category`
SymptomPriority	`primary`, `secondary`	`DiseaseSymptom.priority`
Typicality	`typical`, `possible`, `rare`	`DiseaseSymptom.typicality`
RiskDirection	`risk`, `protective`	`DiseaseRiskFactor.direction`

Tables

Disease

Stores the primary disease entities.

Column	Type	Constraints	Description
disease_id	SERIAL	PK	Internal identifier

Column	Type	Constraints	Description
code	VARCHAR(10)	UNIQUE, NOT NULL	Disease code (documented as ICD-10 in API schema)
name	TEXT	NOT NULL	Disease name (default locale)
description	TEXT	NULLABLE	Disease description
prevention	TEXT	NULLABLE	Prevention recommendations

Relations: - 1:N to DiseaseSymptom - 1:N to DiseaseRiskFactor - 1:N to DiseaseTranslation

---

Symptom

Stores standardized symptoms/signs.

Column	Type	Constraints	Description
symptom_id	SERIAL	PK	Internal identifier
code	VARCHAR(10)	UNIQUE, NULLABLE	Optional code (used in API docs as "SNOMED code")
term	TEXT	UNIQUE, NOT NULL	Canonical symptom term (default locale)
category	SymptomCategory	NULLABLE	`sign` or `symptom`

Relations: - 1:N to DiseaseSymptom - 1:N to SymptomTranslation

---

RiskFactor

Stores standardized risk factors and protective factors.

Column	Type	Constraints	Description
risk_factor_id	SERIAL	PK	Internal identifier
code	VARCHAR(10)	UNIQUE, NULLABLE	Optional code
name	TEXT	NOT NULL	Risk factor name (default locale)
definition	TEXT	NULLABLE	Definition/description

Relations: - 1:N to DiseaseRiskFactor - 1:N to RiskFactorTranslation

---

DiseaseSymptom

Join table connecting diseases and symptoms, with extra attributes for clinical interpretation.

Column	Type	Constraints	Description
disease_symptom_id	SERIAL	PK	Internal identifier
disease_id	INTEGER	FK → Disease(disease_id), NOT NULL, ON DELETE CASCADE	Disease reference
symptom_id	INTEGER	FK → Symptom(symptom_id), NOT NULL, ON DELETE CASCADE	Symptom reference
priority	SymptomPriority	NOT NULL	`primary` or `secondary`
typicality	Typicality	NOT NULL, DEFAULT `typical`	`typical`, `possible`, or `rare`

Constraints / Indexes: - UNIQUE(disease_id, symptom_id) - INDEX(disease_id) name: `idx_disease_symptoms_d` - INDEX(symptom_id) name: `idx_disease_symptoms_s`

DiseaseRiskFactor

Join table connecting diseases and risk factors, with direction (risk vs protective).

Column	Type	Constraints	Description
disease_risk_factor_id	SERIAL	PK	Internal identifier
disease_id	INTEGER	FK → Disease(disease_id), NOT NULL, ON DELETE CASCADE	Disease reference
risk_factor_id	INTEGER	FK → RiskFactor(risk_factor_id), NOT NULL, ON DELETE CASCADE	Risk factor reference
direction	RiskDirection	NOT NULL, DEFAULT `risk`	`risk` or `protective`

Constraints / Indexes: - UNIQUE(disease_id, risk_factor_id) - INDEX(disease_id) name: `idx_disease_risk_d` - INDEX(risk_factor_id) name: `idx_disease_risk_rf`

Source

Stores dataset / organization sources.

Column	Type	Constraints	Description
source_id	SERIAL	PK	Internal identifier
name	TEXT	UNIQUE, NOT NULL	Organization/source name
link	TEXT	UNIQUE, NOT NULL	Source URL

---

disease_translation

Localized fields for Disease per locale.

Column	Type	Constraints	Description
id	SERIAL	PK	Internal identifier
disease_id	INTEGER	FK → Disease(disease_id), NOT NULL, ON DELETE CASCADE	Disease reference
locale	Locale	NOT NULL	`en` or `ru`
name	TEXT	NULLABLE	Localized disease name
description	TEXT	NULLABLE	Localized description
prevention	TEXT	NULLABLE	Localized prevention

Constraints: - UNIQUE(disease_id, locale)

---

symptom_translation

Localized fields for Symptom per locale.

Column	Type	Constraints	Description
id	SERIAL	PK	Internal identifier

Column	Type	Constraints	Description
symptom_id	INTEGER	FK → Symptom(symptom_id), NOT NULL, ON DELETE CASCADE	Symptom reference
locale	Locale	NOT NULL	`en` or `ru`
term	TEXT	NOT NULL	Localized term

Constraints: - UNIQUE(symptom_id, locale)

---

risk_factor_translation

Localized fields for RiskFactor per locale.

Column	Type	Constraints	Description
id	SERIAL	PK	Internal identifier
risk_factor_id	INTEGER	FK → RiskFactor(risk_factor_id), NOT NULL, ON DELETE CASCADE	Risk factor reference
locale	Locale	NOT NULL	`en` or `ru`
name	TEXT	NULLABLE	Localized name
definition	TEXT	NULLABLE	Localized definition

Constraints: - UNIQUE(risk_factor_id, locale)

Relationships

Relationship	Type	Description
Disease → DiseaseSymptom	One-to-Many	A disease has many symptom links (with priority/typicality)
Symptom → DiseaseSymptom	One-to-Many	A symptom can be linked to many diseases
Disease → DiseaseRiskFactor	One-to-Many	A disease has many risk-factor links (with direction)
RiskFactor → DiseaseRiskFactor	One-to-Many	A risk factor can be linked to many diseases

Relationship	Type	Description
Disease → disease_translation	One-to-Many	Localized disease fields per locale
Symptom → symptom_translation	One-to-Many	Localized symptom term per locale
RiskFactor → risk_factor_translation	One-to-Many	Localized risk factor fields per locale

Migrations

Migrations are stored in `backend/prisma/migrations/`.

Version	Folder	Description
001	20251125210618_disease_init	Initial schema: diseases, symptoms, risk factors, join tables, translations, sources, and enums

Seeding

Run migrations and seed data from the backend folder:

```
# from backend/
npm run migrate
npm run generate
npm run seed
```

Notes: - `npm run seed` runs `ts-node prisma/seed.ts`. - The seed script populates diseases, symptoms, risk factors, and sources (see `backend/prisma/seeder/`).

Glossary

Term	Definition
Cardiovascular Disease (CVD)	A class of diseases affecting the heart and blood vessels (used as the core content domain of the platform).
Disease card	A standardized, structured entry for a disease (name, description, symptoms, prevention, and references/analytics when available).

Term	Definition
Symptom	A patient-reported or clinician-observed manifestation linked to a disease (stored as `Symptom` and related via `DiseaseSymptom`).
Risk factor	A factor associated with increased probability of developing a disease (stored as `RiskFactor` and related via `DiseaseRiskFactor`).
Protective factor	A factor associated with reduced probability of developing a disease (modeled as `RiskDirection = protective`).
Primary symptom	A key/core symptom for a disease in this project's data model (modeled as `SymptomPriority = primary`).
Secondary symptom / factor	A non-core or supporting symptom/factor linked to a disease (modeled as `SymptomPriority = secondary`).
Typicality	How common/representative a symptom is for a disease in this project's model (`typical`, `possible`, `rare`).
Prevalence	The proportion of a population with a condition at a given time; used for dataset-driven analytics and comparisons.
Localization / Locale	Supporting multiple languages in UI/content; this project uses `en` and `ru` locales.
MVP	Minimum Viable Product; the smallest scope that delivers core value for the diploma project.
In-scope / Out-of-scope	Features explicitly included vs excluded from the diploma MVP to keep expectations clear.

Acronyms

Acronym	Full Form	Description
API	Application Programming Interface	Backend endpoints exposed under `/api` (e.g., diseases, risk factors, sources).
OpenAPI	OpenAPI Specification	Machine-readable API contract (Swagger) generated for the backend.
Swagger	Swagger UI / OpenAPI docs	Interactive documentation for the API (commonly served at `/api-docs`).
UI	User Interface	The visible web interface (React frontend).

Acronym	Full Form	Description
UX	User Experience	How users interact with and perceive the product's usability.
CVD	Cardiovascular Disease	Domain abbreviation used across docs and code.
ICD-10	International Classification of Diseases, 10th Revision	Standard disease coding system; disease `code` is documented as ICD-10 in API schema.
SNOMED CT	Systematized Nomenclature of Medicine—Clinical Terms	Clinical terminology system; symptom `code` is documented as SNOMED code in API schema (project stores short codes).
ETL	Extract, Transform, Load	Data processing pipeline for cleaning/aggregating datasets before use in analytics/storage.
DB	Database	PostgreSQL used for persistence.
ORM	Object-Relational Mapping	Prisma is used as the ORM to access PostgreSQL.
i18n	Internationalization	Framework/process to support multiple languages (EN/RU in this project).
JWT	JSON Web Token	Token format typically used for auth; mentioned in architecture notes as a possible mechanism.
GDPR	General Data Protection Regulation	EU privacy regulation; project is designed to avoid collecting PII/PHI in the MVP.
PII	Personally Identifiable Information	Personal data that can identify a person (not intended to be collected/processed).
PHI	Protected Health Information	Health data linked to an identifiable person (not intended to be collected/processed).
p95	95th percentile	Performance metric (e.g., p95 API response time).

Acronym	Full Form	Description
LCP	Largest Contentful Paint	Frontend performance metric for perceived load speed.
SVG	Scalable Vector Graphics	Format used for deterministic, cacheable chart embeds (analytics outputs).
WHO	World Health Organization	Example of an official data source; also seeded as a `Source`.
CDC	Centers for Disease Control and Prevention	Example of an official data source; CDC WONDER is seeded as a `Source`.
EU	European Union	Context for GDPR and for sources like Eurostat.
NFR	Non-Functional Requirement	Quality attribute requirement (e.g., performance, security, availability).
FR	Functional Requirement	Behavior/capability the system must provide (e.g., search, filters, browse cards).
KPI	Key Performance Indicator	Metric used to evaluate goals/success (e.g., response time targets).
SEO	Search Engine Optimization	Techniques to improve discoverability (e.g., sitemap/meta tags).
BI	Business Intelligence	Analytics/reporting tooling category (not part of the MVP BI stack).
CSV	Comma-Separated Values	Common dataset file format used by analysis/ETL scripts.

Domain-Specific Terms

Clinical & Epidemiology

Term	Definition
Prevention	Practical guidance intended to reduce disease risk or complications (stored as `Disease.prevention`, can be localized).
Sign	An objective clinical finding (modeled in DB as `SymptomCategory = sign`).
Symptom (category)	A subjective experience reported by a patient (modeled in DB as `SymptomCategory = symptom`).
Risk direction	Indicates whether a factor increases risk or is protective (`RiskDirection = risk | protective`).
Data source	An organization/dataset provider referenced by the platform (stored as `Source` with `name` and `link`).
Evidence-based reference	Content intended to be traceable to reputable sources (e.g., official datasets, clinical guidelines).
Disclaimer (medical)	Statement that the platform provides informational content and is not medical advice/diagnosis.
Primary vs secondary factors	The project's modeling choice to separate core symptoms from additional symptom-related factors for clarity and analysis.

Data & Platform Engineering

Term	Definition
Pagination	Returning results in pages to control payload size; backend uses `skip` (offset) and `take` (limit).
Filter	Narrowing results using query parameters (e.g., `symptom`, `riskFactor`, `search`, `letter`).
Full-text search (project)	Free-text query over diseases/symptoms/risk factors via the `search` query parameter.
Cache	Temporary storage to speed up repeated reads; backend uses Redis (`ioredis`).
CORS	Cross-Origin Resource Sharing
Express	Express.js
Vite	Vite
DI (Inversify)	Dependency Injection
E2E tests	End-to-End tests
Unit tests	Unit tests
Storybook	Storybook

Term	Definition
Migration	Versioned DB change managed by Prisma (`prisma migrate dev`) and stored in `backend/prisma/migrations/`.
Seeding	Loading initial/test data into the DB; backend uses `npm run seed` (TypeScript seed entrypoint).
Prisma Client	Generated TypeScript client used to query the DB in repositories/services.
Docker Compose	Tooling to orchestrate multi-container local environments (API + PostgreSQL + Redis).
Observability	Ability to understand system behavior via logs/metrics/traces; backend integrates logging and Sentry.